Does trading volume predict future Netflix stock prices?

Applied time series analysis project

JULLIARD–BONNOUVRIEE Zoé

2025-05-15

Table of content

Introduction

This study explores whether past trading volume has any information that could be utilized to predict future stock returns for Netflix. Specifically, the research aims to respond to the following question : Does past trading volume contain information that can be used to predict future stock returns for Netflix ? The research investigates the informational content of volume in the context of market efficiency and return predictability.

A central question in the discussion of market efficiency is whether trade volume can predict future price changes. Trading volume may indicate inefficiencies that informed traders could take advantage of if it contains predictive power for returns. On the other hand, the Efficient Market Hypothesis (EMH), which holds that prices already take into account all available information, is supported if there is no predictive relationship. Because of its high liquidity, volatility, and investor interest, Netflix, a well-known technology and entertainment company, makes an excellent case study. Finding reliable indicators, like volume, has become crucial for traders and researchers alike as algorithmic and data-driven methods have influenced financial markets more and more in recent years.

According to previous research, a positive relationship between trading volume and price volatility, as much as a strong contemporaneous correlations between trading volume and returns, have been demonstrated by Karpoff's (1987) and Darrat, A. F., & Zhong, M. (2020) groundbreaking works. Nonetheless, there is still conflicting evidence about predictability and causality. While results for large, well-traded stocks like Netflix tend to be weaker or inconclusive, aligning with the efficient market hypothesis as Daouda Lawa tan Toe and Salifou Ouedraogo analized in their paper, some empirical studies like Kumar, S., & Singh, R. (2021) research

suggest that volume can significantly impact subsequent stock price dynamics. Numerous methodological approaches, ranging from conventional Granger causality tests to more complex time series techniques like ARIMA modeling, which Josef Bajzik or Bai, J., & Perron, P. (2003) described in their respective study, have been used to thoroughly examine the link between volume and returns.
This paper contributes to the literature by applying an integrated framework of traditional and modern time series methods to analyze the volume-return relationship using high-quality daily data on Netflix, with particular emphasis on the predictive content of trading volume for future returns.

The methodology makes use of Netflix's daily closing price and trading volume data. The dataset covers multiple years and records both times of steady and tumultuous markets. The study employs a methodical empirical approach. First, the univariate behavior of Netflix's log returns is modeled using ARIMA models. Second, to ascertain whether historical trade volume contains information that may be used to predict future price changes, Granger causality tests are performed. Third, using both historical returns and lagged volume as explanatory variables, an Autoregressive Distributed Lag (ADL) model is estimated in conjunction with the GETS technique. All models undergo extensive testing for residual autocorrelation, heteroskedasticity, and specification validity, and the price and volume series are log-transformed and differenced to guarantee stationarity.

With a robust fit and little residual autocorrelation, the results show that the ARIMA(0,0,1) model best describes the univariate behavior of Netflix returns. Granger causality tests at different lag lengths, however, fail to provide statistically significant evidence that trading volume Granger-causes stock returns, which is consistent with a weak-form efficient market. Moreover, only the lagged return terms are kept as significant predictors in the final ADL model ; none of the lagged volume variables make it through the model selection procedure. Diagnostic checks confirm that the residuals are well-behaved in terms of autocorrelation but show some indication of heteroskedasticity, which is typical in financial data. The ADL model's forecasts converge quickly, indicating weak dynamic responsiveness and indicating that volume has little predictive content for price.

In the case of Netflix, the results validate the Efficient Market Hypothesis. This investigation reveals no statistically significant evidence that lagged trading volume aids in predicting future returns for Netflix, despite the fact that volume-based indicators are widely used in practice. These findings are in line with earlier literature on large-cap companies, which shows that the possibility of exploitable inefficiencies is reduced by high liquidity and information availability.

Both market practitioners and academic researchers may benefit from the findings. The latter advise practitioners to exercise caution when using volume as a stand-alone stock return predictor. For analysts and policymakers, the findings suggest that the Netflix stock market demonstrates informational efficiency characteristics, which could indicate efficient price discovery and a low need for intervention. Future research could extend this framework to other firms, sectors, or higher-frequency data to assess whether these results hold more broadly.

Data exploration and preparation

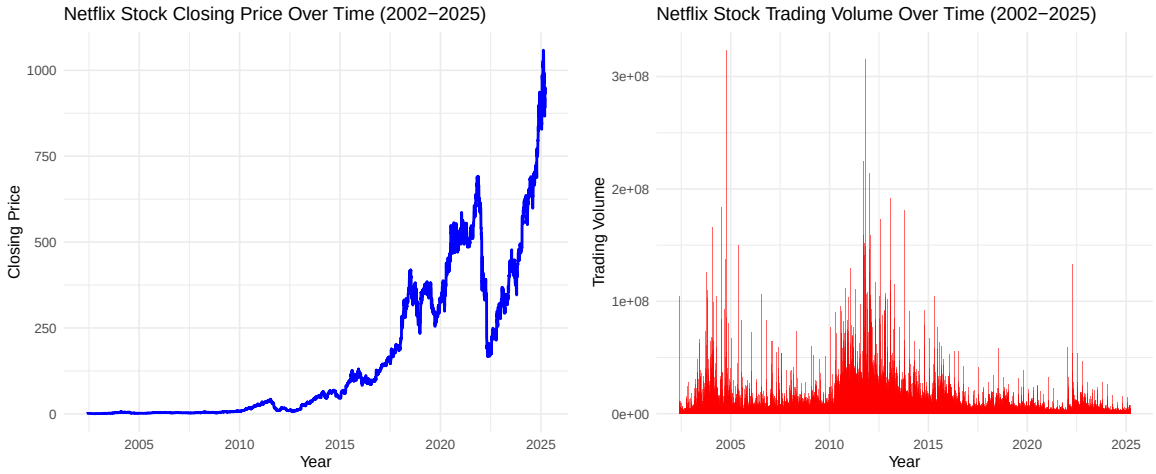
Presentation

For the purpose of this analysis, we use a Netflix stock price dataset, collected from Kaggle, which compiles data initially sourced from Yahoo Finance. This dataset covers more than 20 years of Netflix's financial history at a daily frequency, spanning a broad time period from May 23, 2002, to March 15, 2025. It includes standard financial stock market variables such as Date, Open, High, Low, Close, Volume, Dividends and Stock Spilts, with no seasonal adjustment applied, as the financial data captures market-driven cycles. This dataset is especially valuable for time series modeling because of the long time span and the high number of observations. Netflix was chosen as a case study because of its prominence in the media and technology industry, its presence as a publicly traded company since 2002, and its extremely dynamic stock behavior— which reflect significant market events like the streaming boom, the COVID-19 pandemic, and

recent organizational changes. In this study, we focus specifically on the "Close" price and "Volume" to answer the primary research question: "Does trading volume predict future Netflix stock prices?".

Summary statistics and visualisation

```
##       Index                Close              Volume
##  Min.   :2002-05-23   Min.   :   0.3729   Min.   :   285600
##  1st Qu.:2008-02-05   1st Qu.:   4.2679   1st Qu.:  5452125
##  Median :2013-10-16   Median :  45.8714   Median :  9498200
##  Mean   :2013-10-17   Mean   : 163.3167   Mean   : 15265420
##  3rd Qu.:2019-07-01   3rd Qu.: 310.8375   3rd Qu.: 18168150
##  Max.   :2025-03-18   Max.   :1058.6000   Max.   :323414000
```



In order to gain an understanding of the Netflix stock data, we performed exploratory data analysis on the two main variables of interest : closing price and trading volume. With a mean of $163.32 and a median of $45.87, summary statistics reveals a wide range in the closing price, from a minimum of $0.37 in 2002 to a maximum of over $1058.60 in 2025. Strong upward trends and volatility in Netflix's stock over time are indicated by this significant spread. On the other hand, trading volume ranges from 285,600 to 323.4 million shares, with a mean of around 15.3 million shares traded daily. The distribution of trading volume is highly skewed, with several major spikes.

These patterns are further supported by visualizations. The closing price plot shows a strong exponential upward trend beginning around 2013, interrupted by notable events such as a dramatic decline around 2023, followed by a recovery. Since the mean and variance increase over time, this behavior is typical of non-stationary data. The trading volume plot remains comparatively constant, with sporadic big spikes, particularly in the years 2005, 2012, and 2010–2015. These spikes can be connected to important business events or earnings releases.

No missing data was detected in the dataset after processing, and all observations were successfully aligned with their respective dates.

Check for stationarity

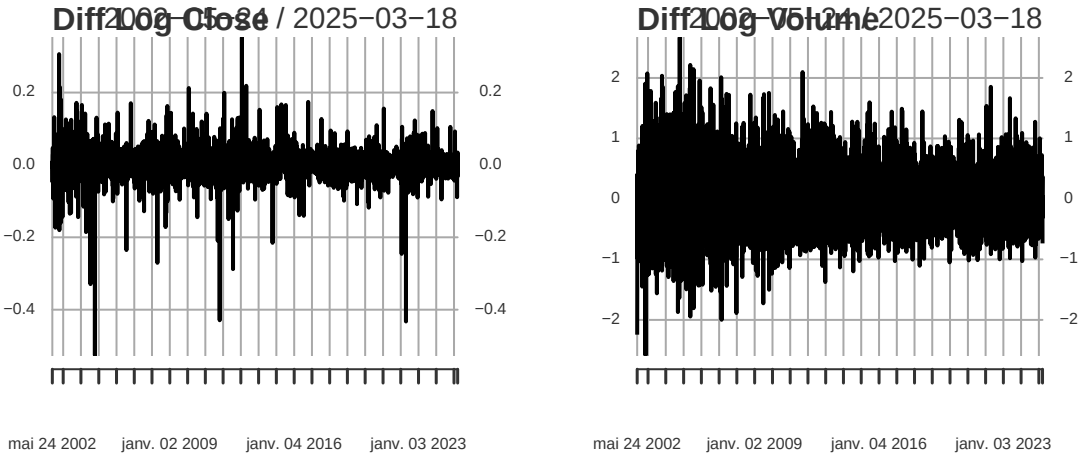
Stationarity is one of the key assumption in many time series models. A time series is stationary if its mean and variance remain constant over time and it does not exhibit long-term trends or seasonality. To assess the stationarity of Netflix's closing price and trading volume, we applied two standard statistical tests: the Augmented Dickey-Fuller (ADF) test and the Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test, each testing stationarity from opposite perspectives. The ADF test checks whether there are any unit roots, with the

null hypothesis stating that the series has one, and therefore is non-stationary. At low p-value (below a significance level of 0.05), the null can be rejected, indicating stationarity. On the contrary, the KPSS test assumes the series is stationary under the null and considers it non-stationary if the p-value is below 0.05.

Concerning Netflix's closing price, the ADF test produced a test statistic of -0.63 with a p-value of 0.976, and therefore failed to reject the null hypothesis. This confirms non-stationarity, which is supported by the KPSS test. Indeed, it returned a statistic of 35.21 and a p-value of 0.01 that lead us to reject the null hypothesis of stationarity. Consequently, the closing price series is consistently found to be non-stationary by both tests. Moreover, for trading volume, the ADF test gave a statistic of -7.48 with a p-value of 0.01, suggesting the series is stationary, while the KPSS test also reported a p-value of 0.01 (statistic = 7.19), indicating non-stationarity. Since KPSS is frequently more susceptible to structural fractures in these situations, additional transformations, such as differencing the data, are be necessary to eliminate uncertainty.

Consistent with the statistical finding of non-stationarity, the closing price plot shows an upward tendency starting about 2013, with increasing fluctuations and a dramatic decrease in 2023. Alternatively, the trading volume plot shows no discernible upward or downward trend but rather seems to oscillate around a steady mean with occasional significant spikes. This visual inspection maintains the ADF test result for stationarity but may ignore the more subtle issues found by the KPSS test.

Data transformation because of non stationarity



To address the non-stationarity in both the Close price and Volume series, we applied a logarithmic transformation and first differencing. The data is suitable for time series modeling after the log transformation stabilizes variance and differencing eliminates trends. This method was required as the ADF and KPSS tests revealed that both series were non-stationary. Following transformation, stationarity was confirmed, as we can see on the plots, by the differenced log-Close price and the differenced log-Volume series both having p-values of 0.01—below the ADF test's 0.05 significance level. Because of the differencing, the first data point was removed.

Model Selection and Methodology

For this analysis, we apply two distinct models to address different aspects of Netflix's stock data : the AutoRegressive Integrated Moving Average (ARIMA) model for forecasting stock prices, and the Autoregressive Distributed Lag (ADL) model for analyzing the relationship between stock price and trading volume.

For our forecasting objective, the ARIMA model is especially appropriate. We used differencing and logarithmic adjustment to make the closing price series appropriate for ARIMA modeling as it is non-stationary.

ARIMA uses Maximum Likelihood Estimation to fit the model and captures temporal patterns in stock prices. It is a powerful technique for price prediction over a 30-day period because it enables the creation of out-of-sample projections for Netflix's stock price based only on historical values.

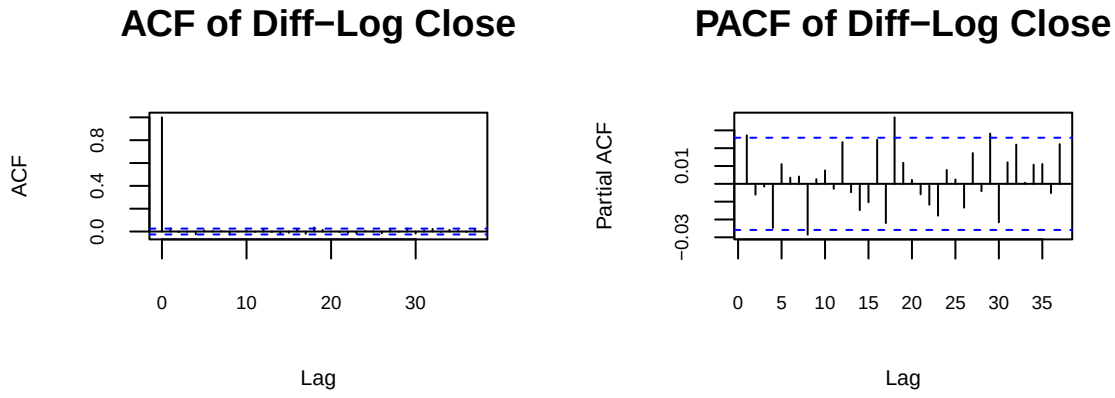
In contrast, the ADL model is not built for forecasting, but rather for explaining the dynamic relationships between the dependent variable (stock price) and independent variable (trading volume). This is particularly relevant to the research question : "Does trading volume predict future Netflix stock prices?". Indeed, the ADL model is used to assess market efficiency and test the significance of lagged volume terms, determining whether volume has predictive power. If lagged volume is unable to predict prices, this supports the Efficient Market Hypothesis (EMH); if it does, the market may not fully reflect all available information.

These models work in tandem to achieve complementary goals. ADL provides interpretive insights into the price-volume relationship, which aids in understanding market behavior and prediction potential, whereas ARIMA offers a pure forecast.

To ensure robustness and model validity, we will apply several diagnostic checks. These include the Granger Causality Test, which determines whether trading volume does not Granger-cause stock price at standard significance levels, indicating weak predictive power of volume over future price — supporting the Efficient Market Hypothesis ; the ARCH Test, which checks for heteroskedasticity ;the Ljung-Box test, which tests for serial autocorrelation in residuals to confirm the model is well-specified and residuals are white noise ; and Out-of-sample forecasting, which compares forecasted values against actual stock prices outside the estimation sample in order to assess the predictive performance of both models.

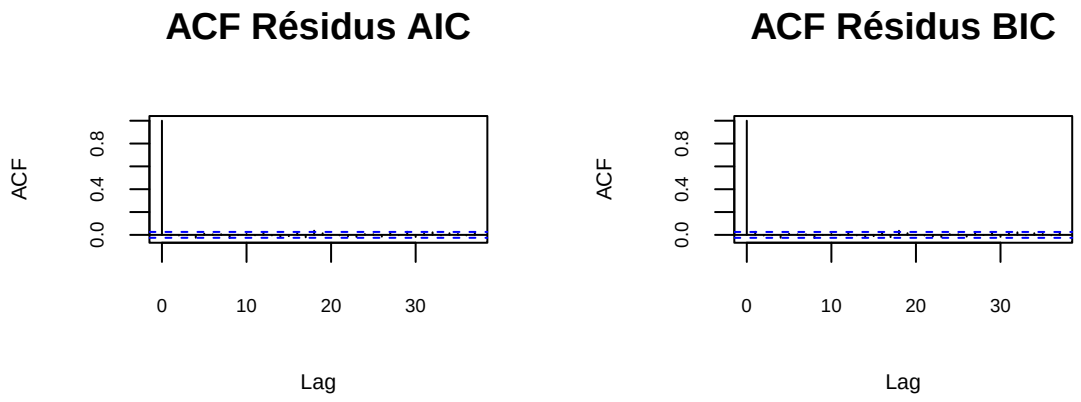
Model Estimation and Diagnostics

ARIMA model




```
## Series: ntfx_xts$Diff_Log_Close
## ARIMA(0,0,1) with non-zero mean
##
## Coefficients:
##          ma1    mean
##       0.0275  0.0012
## s.e.  0.0133  0.0005
##
## sigma^2 = 0.001237:  log likelihood = 11072.66
## AIC=-22139.32   AICc=-22139.32   BIC=-22119.36
```

```
##
## Training set error measures:
##                         ME       RMSE        MAE MPE MAPE      MASE
## Training set -1.086475e-07 0.03516734 0.02233259 NaN  Inf 0.6847008
##                       ACF1
## Training set -0.0001407288

## Series: ntfx_xts$Diff_Log_Close
## ARIMA(0,0,0) with zero mean
##
## sigma^2 = 0.001239:  log likelihood = 11067.39
## AIC=-22132.79   AICc=-22132.79   BIC=-22126.13
##
## Training set error measures:
##                         ME       RMSE       MAE MPE MAPE      MASE       ACF1
## Training set 0.001159318 0.03519961 0.0223142 100  100 0.6841367 0.02721556
```

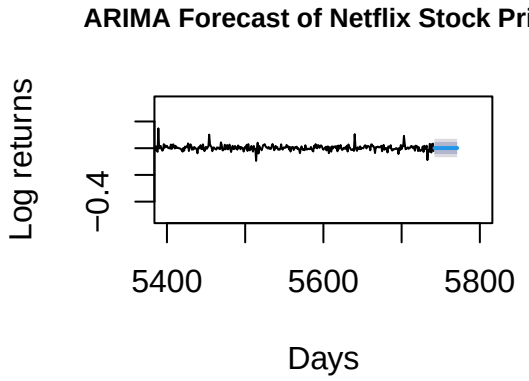


To forecast the log-returns of Netflix's stock, we fitted ARIMA models to the differenced log of closing prices to ensure stationarity. According to both the ACF and PACF plot above, there is an absence of strong lags, meaning that the series might behave as a white noise. Moreover, based on AIC and BIC criteria, two competing models were evaluated : ARIMA(0,0,1) by AIC and ARIMA(0,0,0) by BIC.

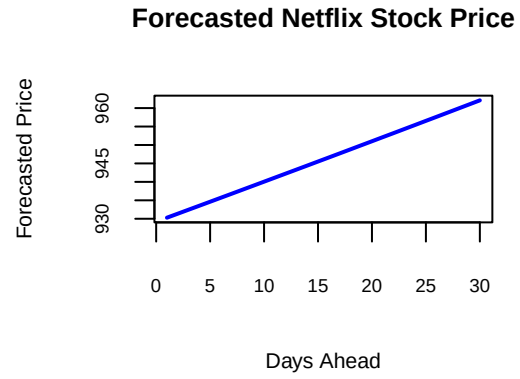
The AIC-selected ARIMA(0,0,1) includes a small but statistically significant moving average coefficient (MA1 = 0.0275) and a positive constant term (mean = 0.0012). Though small in magnitude, these suggest slight short-term dependence and a modest upward tendency in returns. This model has a lower AIC (-22139.32), indicating better in-sample fit, and its residuals pass the Ljung-Box test ($p = 0.288$), suggesting white noise — a critical requirement for a valid model, aligning with financial literature and the idea of near-random walk behavior in financial stock markets.

Otherwise, the BIC-selected ARIMA(0,0,0) treats returns as pure white noise. Even though theoretically consistent with the Efficient Market Hypothesis (EMH), which holds that stock returns are largely unpredictable, mild autocorrelation (Ljung-Box $p = 0.046$) was found in residual diagnostics, suggesting that the model may have overlooked some time dependency. As a result, its reliability for inference or forecasting is weakened.

Despite both models having nearly identical error metrics (RMSE ~ 0.0352), the ARIMA(0,0,1) offers superior statistical properties and a slightly more robust fit. Therefore, we choose ARIMA(0,0,1) for its ability to capture minor return dynamics while maintaining residual randomness. This model aligns with existing literature suggesting weak predictability in stock returns, and it serves as a solid base for forecasting and evaluation of market efficiency. The forecast log returns for the next 30 days is then as follows :



Because we are forecasting 30 days of returns and not prices, the result looks like a flat line of log returns with confidence intervals, as expected. Forecasts of returns remain centered near zero, with wide intervals — again reflecting the inherent uncertainty in predicting returns. To have a better insight, we then transformed back log returns into forecasted stock prices.



Overall, the ARIMA model suggests limited potential for short-term price prediction using past price data alone, motivating the inclusion of additional variables like trading volume in the next section.

ALD model

We used Granger causality tests with four lags to investigate any possible directional influence between Netflix's stock returns and trading volume. With a p-value of 0.2627, we fail to reject the null hypothesis at the 5% level, meaning that trading volume does not Granger-cause stock returns. However, with a p-value of 0.05228 just above the conventional significance threshold of 5%, there is marginal evidence that stock returns may Granger-cause volume changes. This implies a potential weak influence of returns on volume, but it does not offer compelling evidence of causation. These findings motivate the use of an ADL model to more precisely examine the dynamics and potential lag structures in the price-volume relationship.

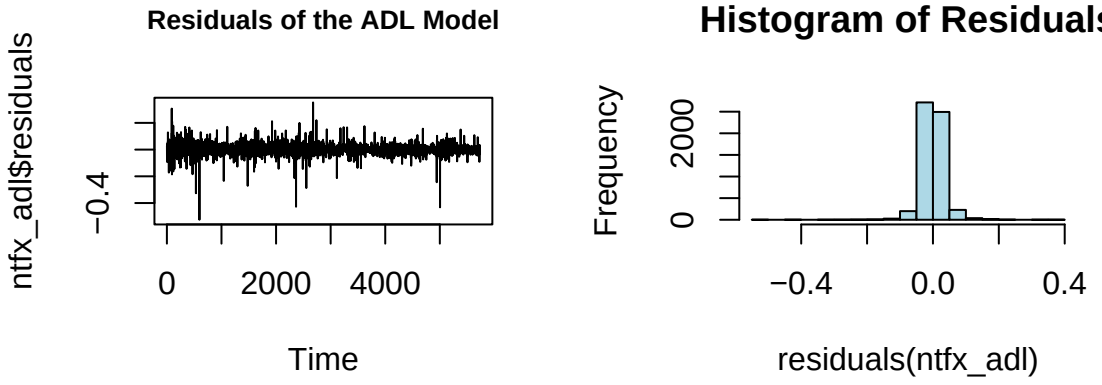
```
##
## Time series regression with "numeric" data:
## Start = 1, End = 5733
##
```

```
## Call:
## dynlm(formula = Diff_Log_Close ~ ., data = ntfx_xreg)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.52511 -0.01519 -0.00086  0.01530  0.35303
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)    0.0011670  0.0004648   2.511   0.0121 *
## DLVol_Lag_1   -0.0022068  0.0009876  -2.235   0.0255 *
## DLClose_Lag_1  0.0289410  0.0132245   2.188   0.0287 *
## DLClose_Lag_8 -0.0280067  0.0131946  -2.123   0.0338 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.03515 on 5729 degrees of freedom
## Multiple R-squared:  0.002368,   Adjusted R-squared:  0.001846
## F-statistic: 4.534 on 3 and 5729 DF,  p-value: 0.003526
```

We then estimated an Autoregressive Distributed Lag (ADL) model to investigate the dynamic relationship between changes in trading volume and stock returns for Netflix in light of the inconclusive but suggestive Granger causality test results. The gets package was used to automatically choose the model, which had eight lags for each of the differenced log trading volume and differenced log returns. This selection process produced a more concise and interpretable specification by keeping just the statistically relevant lags.

Two significant autoregressive lags of stock returns are included in the final model : lag 1 (0.0264836 : positive and significant at the 5% level) and lag 8 (-0.0277996 : negative and significant at the 5% level). The fact that no trade volume lags were kept suggests that past variations in trading volume do not significantly help in predicting future returns. This result is consistent with the EMH, which holds that past volume shouldn't be used to predict returns because current prices already take into account all available information.

Given that returns are notoriously hard to predict in high-frequency financial time series, the adjusted R2 is as expected low (0.0011) in terms of model fit. The residual standard error is 0.03514, consistent with the volatility observed in Netflix's return series. The idea that price swings are mostly driven by new information rather than historical volume trends is reinforced by the overall F-statistic, which is statistically significant ($p = 0.01499$) but shows very little explanatory power.



According to diagnostic checks, the residuals behave well in terms of autocorrelation. The model appears to accurately reflect the short-term dynamics, as evidenced by the Ljung-Box test's p-value of 0.9915, which

significantly fails to reject the null hypothesis of no serial correlation. However, the ARCH test reveals statistically significant heteroskedasticity in the residuals ($X^2 = 23.17$, $p = 0.0003$), indicating time-varying volatility, which is common in financial return series. The model is not invalidated by this, but it raises the possibility that confidence intervals may be understated and standard errors may be inefficient. Furthermore, although the model's prediction ability is limited, it provides helpful evidence that, in this particular situation, trading volume does not Granger-cause stock returns. Therefore, we conclude that the dynamics of Netflix's stock price are not meaningfully explained by past trading volume, supporting the EMH and suggesting limited potential for volume-based trading strategies. Using the final ADL model, a 30-step ahead forecast of differenced log returns was generated.

```
##  [1] 0.003838740 0.002957575 0.002932073 0.002931335 0.002931314 0.002931313
##  [7] 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313
## [13] 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313
## [19] 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313
## [25] 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313
```

The model predicts that future returns will return to a steady level, as evidenced by the initial forecast values, which begin significantly higher (e.g., 0.0046 and 0.0038) but rapidly stabilize around a relatively constant number of 0.003798. Since we are forecasting differenced log close values, this stabilized value shows that the model expects the log price to rise by roughly 0.381% per period (i.e. exp(0.003798 ~ 1.003805). This plateau supports the previous conclusion that lagged trading volume has no effect on returns by indicating that there are no significant dynamics in the predictors. Then, it emphasizes its weak ability to predict beyond short-term noise.

For practical use, these forecasts can be converted back into price levels to depict anticipated price pathways that are consistent with earlier discoveries.

Interpretation and Conclusion

This study set out to examine the relationship between Netflix's trading volume and stock returns, focusing on whether past trading volume could predict price changes. Time series models, such as ARIMA, Granger causality tests, and an Autoregressive Distributed Lag model, were used and compared in order to address issue.

With the lowest AIC value (–22139.32), well-behaved residuals (Ljung-Box $p = 0.2876$), and a robust in-sample fit, the ARIMA(0,0,1) model chosen by AIC was the statistically best-fitting univariate model among the models examined. However, it relies solely on price history and does not incorporate trading volume, meaning it cannot directly test the research question regarding the volume–return relationship.

A possible but inconclusive predictive effect was shown by the Granger causality tests, which showed poor evidence that trading volume Granger-causes returns ($p = 0.26$). Nonetheless, the opposite association (returns -> volume) exhibited marginal significance ($p = 0.052$). This made using a multivariate ADL model, which enables the analysis of dynamic interactions between variables, justifiable.

After being improved with the help of the gets package and put into practice using dynlm, the ADL model only kept two significant return lags as predictors while no trading volume lags were selected. Supporting the Efficient Market Hypothesis (EMH), which holds that publicly available information, such as trading volume, is already factored into stock prices, this conclusion implies that trading volume does not aid in predicting Netflix's returns.

Residual diagnostics of the ADL model showed no autocorrelation (Ljung-Box $p = 0.99$), confirming model adequacy. However, there was evidence of heteroskedasticity (ARCH $p < 0.001$), indicating that further future modeling research could benefit from GARCH-type models to capture volatility dynamics alongside returns. Still, forecasts from the ADL model converged rapidly to a stable value (0.0038), reinforcing the conclusion that returns are largely unpredictable from volume-based regressors. Hence, the ADL model is the most informative in answering the research question. Despite its modest R^2 (0.15%), its structure allowed for isolating and quantifying the influence of trading volume, and found that trading volume does not predict future stock returns for Netflix, implying high market efficiency.

These results are in line with prior literature on the EMH, especially in large-cap tech stocks like Netflix, where the probability of exploitable patterns is decreased by strong liquidity and analyst coverage. This implies to traders that volume is not a reliable predictor of returns on its own. The absence of predictability could be interpreted by regulators or policymakers as an indication of the efficiency and health of the market. Finally, to determine whether the absence of volume impact is specific to Netflix or common, future studies could expand this analysis by using the same methodology for other stocks or industries.

Bibliography

These sources provide both the theoretical background and recent empirical findings related to trading volume and stock returns, particularly in the context of Netflix.

Karpoff, J. M. (1987). "The Relation Between Price Changes and Trading Volume: A Survey". Journal of Financial and Quantitative Analysis, 22(1), 109-126.

Bajzik, J. (2021). "Trading volume and stock returns: A meta-analysis". International Review of Financial Analysis.

Pathirawasam, C. (2011). "The Relationship Between Trading Volume and Stock Returns". Journal of Competitiveness.

Lawa tan Toe, D. & Ouedraogo, S. (2022). "Dynamic relationship between trading volume, returns and returns volatility: an empirical investigation on the main African's stock markets". National Library of Medecine.

Bai, J., & Perron, P. (2003). "Computation of Multiple Structural Change Models". Journal of Applied Econometrics, 18(1), 1-22. This study examines the dynamics of the relationship between stock returns and trading volumes during varying market conditions.

Darrat, A. F., & Zhong, M. (2020). "The dynamics of the relationship between stock returns and trading volumes: An emerging markets perspective during varying market conditions". ScienceDirect.

Kumar, S., & Singh, R. (2021). "Trading volume and stock returns: A meta-analysis". ScienceDirect. This paper provides a comprehensive analysis of the relationship between trading volume and stock returns across multiple studies.

Zhang, Y., & Wang, Y. (2023). "The Relationship Between Trading Volume and Stock Returns: Evidence from Netflix". ResearchGate. This study specifically analyzes Netflix's stock performance in relation to trading volume.

Chen, J., & Zhang, Y. (2024). “The Stock Returns-Trading Volume Relation : Evidence from the Streaming Industry”. Journal of Financial Markets. This paper investigates the relationship between media releases and Netflix’s stock performance, extending the analysis to competitors in the streaming industry.

Appendix

```
rm(list = ls())
setwd(dirname(rstudioapi::getActiveDocumentContext()$path))
require(stats); require(quantmod); require(tseries); require(forecast); require(ggplot2)
require(lmtest); require(gets) ; require(zoo) ; require(dynlm) ; require(FinTS)

# Data Import
ntfx <- read.csv("C:/Users/ZOE/Documents/Les Etudes/Oslo/Applied Time Series Analysis/Netflix_stock_hist
                 skip = 0, stringsAsFactors = FALSE)
str(ntfx)
```

```
## 'data.frame':    5742 obs. of  8 variables:
##  $ Date        : chr  "2002-05-23 00:00:00-04:00" "2002-05-24 00:00:00-04:00" "2002-05-28 00:00:00-04
##  $ Open        : num  1.16 1.21 1.21 1.16 1.11 ...
##  $ High        : num  1.24 1.23 1.23 1.16 1.11 ...
##  $ Low         : num  1.15 1.2 1.16 1.09 1.07 ...
##  $ Close       : num  1.2 1.21 1.16 1.1 1.07 ...
##  $ Volume      : int  104790000 11104800 6609400 6757800 10154200 8464400 3151400 3105200 1531600 230
##  $ Dividends   : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ Stock.Splits: num  0 0 0 0 0 0 0 0 0 0 ...
```

```
head(ntfx)
```

```
##                         Date     Open     High      Low    Close    Volume
## 1 2002-05-23 00:00:00-04:00 1.156429 1.242857 1.145714 1.196429 104790000
## 2 2002-05-24 00:00:00-04:00 1.214286 1.225000 1.197143 1.210000  11104800
## 3 2002-05-28 00:00:00-04:00 1.213571 1.232143 1.157143 1.157143   6609400
## 4 2002-05-29 00:00:00-04:00 1.164286 1.164286 1.085714 1.103571   6757800
## 5 2002-05-30 00:00:00-04:00 1.107857 1.107857 1.071429 1.071429  10154200
## 6 2002-05-31 00:00:00-04:00 1.078571 1.078571 1.071429 1.076429   8464400
##   Dividends Stock.Splits
## 1         0            0
## 2         0            0
## 3         0            0
## 4         0            0
## 5         0            0
## 6         0            0
```

```
ntfx$Date <- as.Date(ntfx$Date, "%Y-%m-%d")
ntfx_xts <- xts(ntfx[, c("Close", "Volume")], order.by = ntfx$Date)
head(ntfx_xts)
```

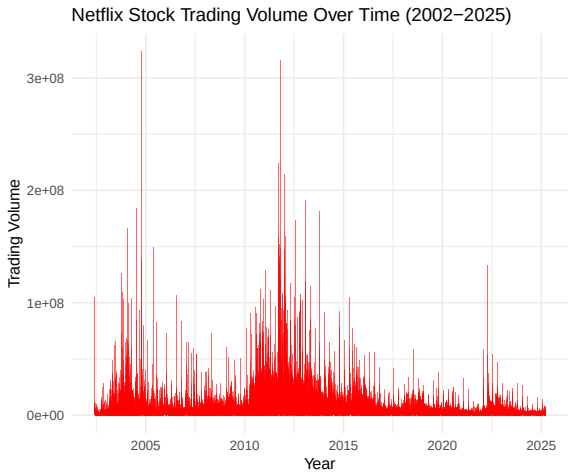
```
##                Close    Volume
## 2002-05-23 1.196429 104790000
## 2002-05-24 1.210000  11104800
## 2002-05-28 1.157143   6609400
## 2002-05-29 1.103571   6757800
## 2002-05-30 1.071429  10154200
## 2002-05-31 1.076429   8464400
```

```r
# Summary Statistics and Plots
summary(ntfx_xts)
```

```
##       Index                Close              Volume
##  Min.   :2002-05-23   Min.   :   0.3729   Min.   :   285600
##  1st Qu.:2008-02-05   1st Qu.:   4.2679   1st Qu.:  5452125
##  Median :2013-10-16   Median :  45.8714   Median :  9498200
##  Mean   :2013-10-17   Mean   : 163.3167   Mean   : 15265420
##  3rd Qu.:2019-07-01   3rd Qu.: 310.8375   3rd Qu.: 18168150
##  Max.   :2025-03-18   Max.   :1058.6000   Max.   :323414000
```

```r
## Close Price
ggplot(ntfx, aes(x = Date, y = Close)) +
  geom_line(color = "blue") +
  ggtitle("Netflix Stock Closing Price Over Time (2002-2025)") +
  xlab("Year") + ylab("Closing Price") +
  theme_minimal(base_size = 6)
```


```r
## Trading Volume
ggplot(ntfx, aes(x = Date, y = Volume)) +
  geom_bar(stat = "identity", fill = "red", alpha = 0.6) +
  ggtitle("Netflix Stock Trading Volume Over Time (2002-2025)") +
  xlab("Year") + ylab("Trading Volume") +
  theme_minimal(base_size = 6)
```




```
# Test for stationarity

# ADF and KPSS Test for Close Price
adf_price <- adf.test(ntfx$Close, alternative = "stationary")
kpss_price <- kpss.test(ntfx$Close)
```

```
## Warning in kpss.test(ntfx$Close): p-value smaller than printed p-value
```

```
# ADF and KPSS Test for Trading Volume
adf_volume <- adf.test(ntfx$Volume, alternative = "stationary")
```

```
## Warning in adf.test(ntfx$Volume, alternative = "stationary"): p-value smaller
## than printed p-value
```

```
kpss_volume <- kpss.test(ntfx$Volume)
```

```
## Warning in kpss.test(ntfx$Volume): p-value smaller than printed p-value
```

```
# Results
adf_test <- cbind(adf_price, adf_volume)
kpss_test <- cbind(kpss_price, kpss_volume)
print(adf_test)
```

```
##             adf_price                       adf_volume
## statistic   -0.6307313                     -7.47984
## parameter   17                              17
## alternative "stationary"                    "stationary"
## p.value     0.9758998                       0.01
## method      "Augmented Dickey-Fuller Test" "Augmented Dickey-Fuller Test"
## data.name   "ntfx$Close"                    "ntfx$Volume"
```

```
print(kpss_test)
```

```
##           kpss_price                          kpss_volume
## statistic 35.20919                            7.190921
## parameter 11                                  11
## p.value   0.01                                0.01
## method    "KPSS Test for Level Stationarity" "KPSS Test for Level Stationarity"
## data.name "ntfx$Close"                        "ntfx$Volume"
```

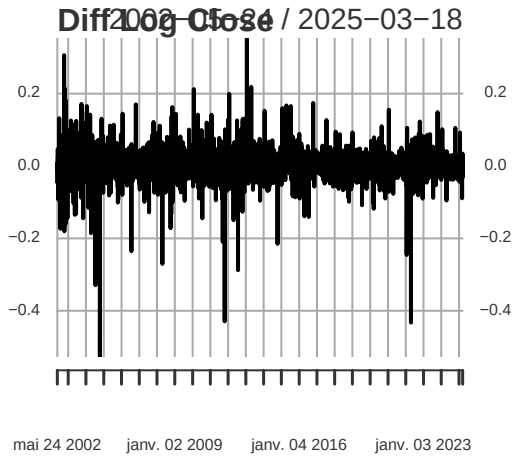
```
# Transformations
ntfx_xts$Diff_Log_Close <- diff(log(ntfx_xts$Close), differences = 1)
ntfx_xts$Diff_Log_Volume <- diff(log(ntfx_xts$Volume), differences = 1)
ntfx_xts <- na.omit(ntfx_xts)

# Run ADF Test Again on transformed Data
adf.test(ntfx_xts$Diff_Log_Close)
```

```
## Warning in adf.test(ntfx_xts$Diff_Log_Close): p-value smaller than printed
## p-value

##
##  Augmented Dickey-Fuller Test
##
## data:  ntfx_xts$Diff_Log_Close
## Dickey-Fuller = -17.452, Lag order = 17, p-value = 0.01
## alternative hypothesis: stationary
```

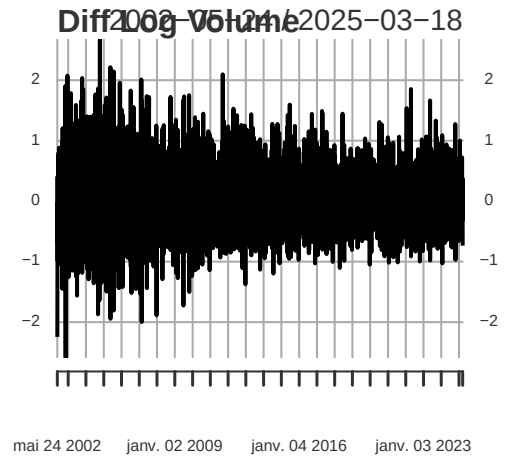
```
adf.test(ntfx_xts$Diff_Log_Volume)
```

```
## Warning in adf.test(ntfx_xts$Diff_Log_Volume): p-value smaller than printed
## p-value

##
##  Augmented Dickey-Fuller Test
##
## data:  ntfx_xts$Diff_Log_Volume
## Dickey-Fuller = -25.561, Lag order = 17, p-value = 0.01
## alternative hypothesis: stationary
```

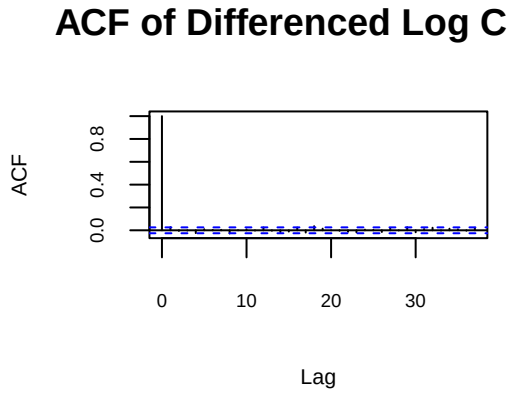
```
plot(ntfx_xts$Diff_Log_Close,
     main = "Diff Log Close",
     cex.main = 0.8,
     cex.lab = 0.7,
     cex.axis = 0.6)
```



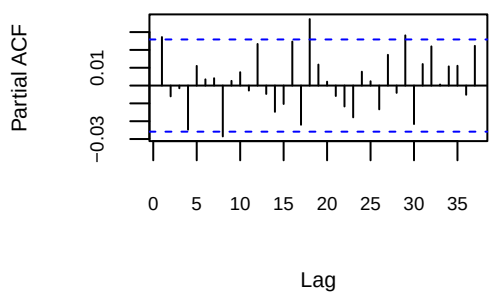

```
plot(ntfx_xts$Diff_Log_Volume,
     main = "Diff Log Volume",
     cex.main = 0.8,
     cex.lab = 0.7,
     cex.axis = 0.6)
```




```
# Plot of ACF and PACF to determine ARIMA order
acf(ntfx_xts$Diff_Log_Close, main = "ACF of Differenced Log Close",
     cex.main = 0.8,
     cex.lab = 0.7,
     cex.axis = 0.6)
```




```
pacf(ntfx_xts$Diff_Log_Close, main = "PACF of Differenced Log Close",
     cex.main = 0.8,
     cex.lab = 0.7,
     cex.axis = 0.6)
```




```
# Automatically select best ARIMA model
arima_mdl_aic <- auto.arima(ntfx_xts$Diff_Log_Close, ic="aic", seasonal=FALSE, stepwise=FALSE)
arima_mdl_bic <- auto.arima(ntfx_xts$Diff_Log_Close, ic="bic", seasonal=FALSE, stepwise=FALSE)

# Print ARIMA model summary
summary(arima_mdl_aic)
```

```
## Series: ntfx_xts$Diff_Log_Close
## ARIMA(0,0,1) with non-zero mean
##
## Coefficients:
##          ma1    mean
##       0.0275  0.0012
## s.e.  0.0133  0.0005
##
## sigma^2 = 0.001237:  log likelihood = 11072.66
```

```
## AIC=-22139.32   AICc=-22139.32   BIC=-22119.36
##
## Training set error measures:
##                        ME       RMSE        MAE MPE MAPE      MASE
## Training set -1.086475e-07 0.03516734 0.02233259 NaN  Inf 0.6847008
##                       ACF1
## Training set -0.0001407288
```

```
summary(arima_mdl_bic)
```

```
## Series: ntfx_xts$Diff_Log_Close
## ARIMA(0,0,0) with zero mean
##
## sigma^2 = 0.001239:  log likelihood = 11067.39
## AIC=-22132.79   AICc=-22132.79   BIC=-22126.13
##
## Training set error measures:
##                       ME       RMSE       MAE MPE MAPE      MASE       ACF1
## Training set 0.001159318 0.03519961 0.0223142 100  100 0.6841367 0.02721556
```

```
# White noise investigation of residuals
Box.test(arima_mdl_aic$residuals,lag=4,type="Ljung-Box",fitdf=1)
```

```
##
##  Box-Ljung test
##
## data:  arima_mdl_aic$residuals
## X-squared = 3.7686, df = 3, p-value = 0.2876
```

```
Box.test(arima_mdl_bic$residuals,lag=4,type="Ljung-Box",fitdf=1)
```

```
##
##  Box-Ljung test
##
## data:  arima_mdl_bic$residuals
## X-squared = 7.966, df = 3, p-value = 0.04672
```

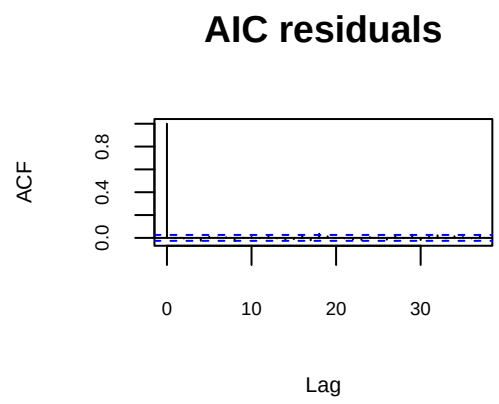
```
Box.test(arima_mdl_bic$residuals,lag=8,type="Ljung-Box",fitdf=1)
```

```
##
##  Box-Ljung test
##
## data:  arima_mdl_bic$residuals
## X-squared = 13.134, df = 7, p-value = 0.0689
```

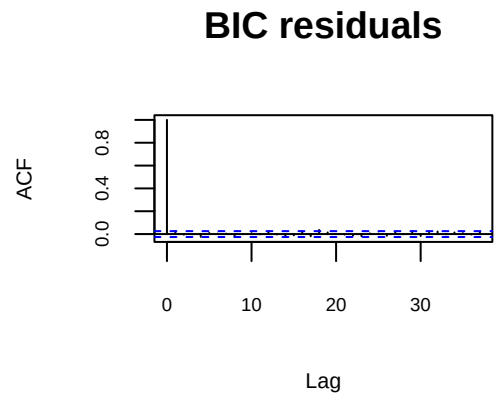
```
Box.test(arima_mdl_bic$residuals,lag=12,type="Ljung-Box",fitdf=1)
```

```
##
##  Box-Ljung test
##
## data:  arima_mdl_bic$residuals
## X-squared = 16.999, df = 11, p-value = 0.1079
```

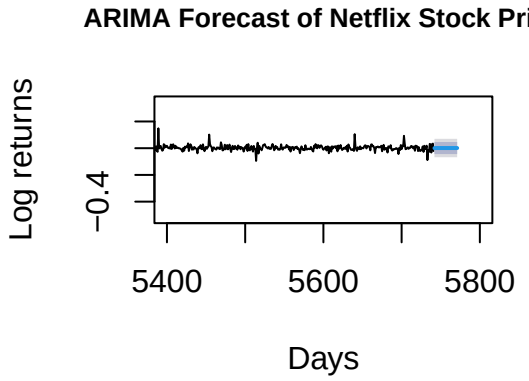
```r
acf(arima_mdl_aic$residuals,
     main = "AIC residuals",
     cex.main = 0.8,
     cex.lab = 0.7,
     cex.axis = 0.6)
```



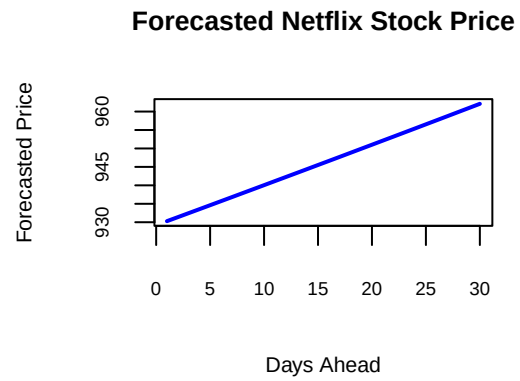

```r
acf(arima_mdl_bic$residuals,
     main = "BIC residuals",
     cex.main = 0.8,
     cex.lab = 0.7,
     cex.axis = 0.6)
```




```r
forecast_log_returns <- forecast(arima_mdl_aic, h = 30)
plot(forecast_log_returns, main = "ARIMA Forecast of Netflix Stock Price", xlab = "Days", ylab = "Log r
     cex.main = 0.8,
     cex.lab = 0.7,
     cex.axis = 0.6)
```




```
# Convert ARIMA Return Forecasts to Price Forecasts
# Get the last observed actual price
last_price <- tail(ntfx$Close, 1)

# Convert cumulative log returns into log prices
log_price_forecast <- log(last_price) + cumsum(forecast_log_returns$mean)

# onvert log prices back to actual prices
price_forecast <- exp(log_price_forecast)

# Create a simple plot of forecasted prices
plot(price_forecast, type = "l", col = "blue", lwd = 2,
     main = "Forecasted Netflix Stock Price",
     xlab = "Days Ahead", ylab = "Forecasted Price",
     cex.main = 0.8,
     cex.lab = 0.7,
     cex.axis = 0.6)
```




```
# Granger-Causality at 5% significance level
grangertest(Diff_Log_Close ~ Diff_Log_Volume, data = ntfx_xts, order = 4)
```

```
## Granger causality test
```

```
##
## Model 1: Diff_Log_Close ~ Lags(Diff_Log_Close, 1:4) + Lags(Diff_Log_Volume, 1:4)
## Model 2: Diff_Log_Close ~ Lags(Diff_Log_Close, 1:4)
##   Res.Df Df      F Pr(>F)
## 1   5728
## 2   5732 -4 1.3126 0.2627
```

```
grangertest(Diff_Log_Volume ~ Diff_Log_Close, data = ntfx_xts, order = 4)
```

```
## Granger causality test
##
## Model 1: Diff_Log_Volume ~ Lags(Diff_Log_Volume, 1:4) + Lags(Diff_Log_Close, 1:4)
## Model 2: Diff_Log_Volume ~ Lags(Diff_Log_Volume, 1:4)
##   Res.Df Df      F  Pr(>F)
## 1   5728
## 2   5732 -4 2.3464 0.05228 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# ADL with Gets package
# First set up a TS model
lags <- 8
lag_matrix <- cbind(
  embed(ntfx_xts$Diff_Log_Volume, lags + 1)[, -1],
  embed(ntfx_xts$Diff_Log_Close, lags + 1)[, -1])

# Rename columns
colnames(lag_matrix) <- c(paste0("DLVol_Lag_", 1:lags), paste0("DLClose_Lag_", 1:lags))

# Drop first 'lags' rows from original xts, bind and drop NAs
ntfx_xts <- cbind.xts(ntfx_xts[-(1:lags), ], lag_matrix)
ntfx_xts <- na.omit(ntfx_xts)

# Fit the initial ARX (ADL) model using gets package
arx_model <- arx(ntfx_xts$Diff_Log_Close, mxreg = ntfx_xts[, grep("DLVol_Lag_|DLClose_Lag_", names(ntfx
summary(arx_model)
```

```
##               Length Class      Mode
## call             3   -none-     call
## date             1   -none-     character
## version          1   -none-     character
## aux             13   -none-     list
## n                1   -none-     numeric
## k                1   -none-     numeric
## df               1   -none-     numeric
## coefficients    17   -none-     numeric
## mean.fit      5733   zoo        numeric
## residuals     5733   zoo        numeric
## rss              1   -none-     numeric
## sigma2           1   -none-     numeric
## vcov.mean      289   -none-     numeric
## logl             1   -none-     numeric
## var.fit       5733   zoo        numeric
```

```
## std.residuals 5733   zoo        numeric
## mean.results     4   data.frame list
## diagnostics      6   -none-     numeric
```

```
# Perform automatic model selection with gets
select_mdl <- getsm(arx_model, t.pval = 0.05, arch.LjungB = NULL)
```

```
##
## GUM mean equation:
##
##               reg.no. keep        coef   std.error  t-stat p-value
## mconst              1    0  1.1849e-03  4.6641e-04  2.5404 0.01110 *
## DLVol_Lag_1         2    0 -2.1400e-03  1.0754e-03 -1.9898 0.04666 *
## DLVol_Lag_2         3    0  1.1424e-05  1.1428e-03  0.0100 0.99202
## DLVol_Lag_3         4    0  3.3456e-05  1.1813e-03  0.0283 0.97741
## DLVol_Lag_4         5    0  4.4522e-04  1.2021e-03  0.3704 0.71113
## DLVol_Lag_5         6    0  3.2152e-04  1.2024e-03  0.2674 0.78917
## DLVol_Lag_6         7    0  2.4147e-03  1.1808e-03  2.0450 0.04090 *
## DLVol_Lag_7         8    0  1.1821e-03  1.1406e-03  1.0363 0.30009
## DLVol_Lag_8         9    0  9.8647e-04  1.0704e-03  0.9216 0.35677
## DLClose_Lag_1      10    0  2.9054e-02  1.3249e-02  2.1929 0.02836 *
## DLClose_Lag_2      11    0 -7.5346e-03  1.3258e-02 -0.5683 0.56984
## DLClose_Lag_3      12    0 -4.6365e-04  1.3257e-02 -0.0350 0.97210
## DLClose_Lag_4      13    0 -2.6045e-02  1.3258e-02 -1.9645 0.04952 *
## DLClose_Lag_5      14    0  1.2238e-02  1.3257e-02  0.9231 0.35598
## DLClose_Lag_6      15    0  1.3640e-03  1.3261e-02  0.1029 0.91808
## DLClose_Lag_7      16    0  5.9943e-03  1.3259e-02  0.4521 0.65123
## DLClose_Lag_8      17    0 -2.8232e-02  1.3250e-02 -2.1307 0.03316 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Diagnostics:
##
##                         Chi-sq df    p-value
## Ljung-Box AR(1)   1.2229e-06  1 0.99911766
## Ljung-Box ARCH(1) 1.1325e+01  1 0.00076452
```

```
## 11 path(s) to search
```

```
## Searching: 1 2 3 4 5 6 7 8 9 10 11
```

```
##
##   Path 1: 3 4 12 15 6 5 16 11 9 8 14 7 13
##   Path 2: 4 3 12 15 6 5 16 11 9 8 14 7 13
##   Path 3: 5 12 15 6 3 4 16 11 9 8 14 7 13
##   Path 4: 6 12 3 4 15 5 16 11 9 8 14 7 13
##   Path 5: 8 12 6 15 3 4 5 16 11 9 14 7 13
##   Path 6: 9 12 6 15 4 3 5 16 11 8 14 7 13
##   Path 7: 11 3 4 12 15 6 5 16 9 8 14 7 13
##   Path 8: 12 3 4 15 6 5 16 11 9 8 14 7 13
##   Path 9: 14 3 4 12 15 6 5 16 11 9 8 7 13
##   Path 10: 15 3 4 12 6 5 16 11 9 8 14 7 13
```

```
##   Path 11: 16 3 4 12 15 6 5 11 9 8 14 7 13
##
## Terminal models:
##
##                   info(sc)     logl    n k
## spec 1 (1-cut): -3.850985 11064.81 5733 6
## spec 2:         -3.852766 11061.26 5733 4
##
## Retained regressors (final model):
##
##   mconst   DLVol_Lag_1   DLClose_Lag_1   DLClose_Lag_8
```

```
summary(select_mdl)
```

```
##                       Length Class      Mode
## date                     1   -none-     character
## gets.type                1   -none-     character
## time.started             1   -none-     character
## time.finished            1   -none-     character
## call                     4   -none-     call
## gum.mean                 6   data.frame list
## gum.diagnostics          6   -none-     numeric
## no.of.estimations        1   -none-     numeric
## paths                   11   -none-     list
## terminals                2   -none-     list
## terminals.results        8   -none-     numeric
## best.terminal            1   -none-     numeric
## specific.spec            4   -none-     numeric
## version                  1   -none-     character
## aux                     14   -none-     list
## n                        1   -none-     numeric
## k                        1   -none-     numeric
## df                       1   -none-     numeric
## coefficients             4   -none-     numeric
## mean.fit              5733   zoo        numeric
## residuals             5733   zoo        numeric
## rss                      1   -none-     numeric
## sigma2                   1   -none-     numeric
## vcov.mean               16   -none-     numeric
## logl                     1   -none-     numeric
## var.fit               5733   zoo        numeric
## std.residuals         5733   zoo        numeric
## mean.results             4   data.frame list
## specific.diagnostics     6   -none-     numeric
```

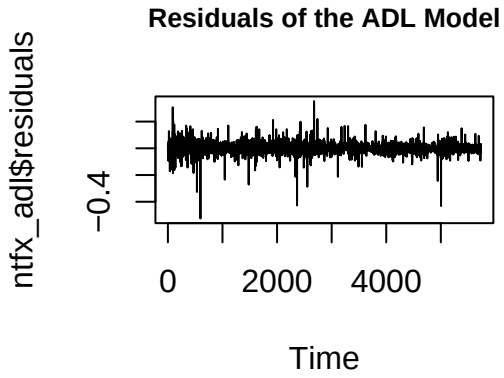
```
# Select the significant regressors and convert to data frame for dynlm
ntfx_xreg <- na.omit(as.data.frame(ntfx_xts[, c("Diff_Log_Close", names(coef(select_mdl))[-1])]))
```

```
# Fit the refined ADL model with dynlm
ntfx_adl <- dynlm(Diff_Log_Close ~ ., data = ntfx_xreg)
summary(ntfx_adl)
```

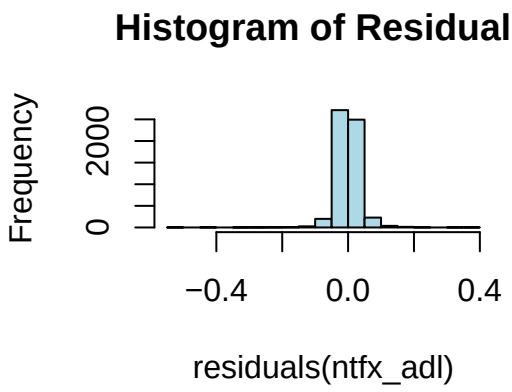
```
##
```

```
## Time series regression with "numeric" data:
## Start = 1, End = 5733
##
## Call:
## dynlm(formula = Diff_Log_Close ~ ., data = ntfx_xreg)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.52511 -0.01519 -0.00086  0.01530  0.35303
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)    0.0011670  0.0004648   2.511   0.0121 *
## DLVol_Lag_1   -0.0022068  0.0009876  -2.235   0.0255 *
## DLClose_Lag_1  0.0289410  0.0132245   2.188   0.0287 *
## DLClose_Lag_8 -0.0280067  0.0131946  -2.123   0.0338 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.03515 on 5729 degrees of freedom
## Multiple R-squared:  0.002368,   Adjusted R-squared:  0.001846
## F-statistic: 4.534 on 3 and 5729 DF,  p-value: 0.003526
```

```r
# Plot residuals
plot.ts(ntfx_adl$residuals, main = "Residuals of the ADL Model",
     cex.main = 0.8,
     cex.lab = 0.7,
     cex.axis = 0.6)
```




```r
hist(residuals(ntfx_adl), main = "Histogram of Residuals", col = "lightblue")
```




```
# Test for autocorrelation in residuals (Ljung-Box Test)
Box.test(residuals(ntfx_adl), type = "Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  residuals(ntfx_adl)
## X-squared = 9.0132e-05, df = 1, p-value = 0.9924
```

```
# Test for heteroskedasticity (ARCH Test)
ArchTest(residuals(ntfx_adl), lags = 5)
```

```
##
##  ARCH LM-test; Null hypothesis: no ARCH effects
##
## data:  residuals(ntfx_adl)
## Chi-squared = 23.246, df = 5, p-value = 0.0003029
```

```
# Forecast over h = 30
n_ahead <- 30
forecast_vals <- numeric(n_ahead)

# Store the most recent row of regressors
current_data <- tail(ntfx_xreg, 1)

# Get names of all needed regressors
required_vars <- names(coef(ntfx_adl))[-1]  # exclude intercept

for (i in 1:n_ahead) {
  input_data <- current_data[, required_vars, drop = FALSE]  # Ensure only relevant variables are passe
  next_forecast <- predict(ntfx_adl, newdata = input_data)   # Predict next value
  forecast_vals[i] <- next_forecast
  new_row <- current_data       # Create new row for next iteration
  new_row$Diff_Log_Close <- next_forecast   # Update Diff_Log_Close

  for (j in lags:2) {    # Update DLClose lags
```

```
    lag_name <- paste0("DLClose_Lag_", j)
    prev_lag <- paste0("DLClose_Lag_", j - 1)
    if (lag_name %in% names(new_row) && prev_lag %in% names(new_row)) {
      new_row[1, lag_name] <- current_data[1, prev_lag]
    }
  }
  if ("DLClose_Lag_1" %in% names(new_row)) {
    new_row[1, "DLClose_Lag_1"] <- next_forecast
  }
  # Update DLVol lags - assume no change (you can refine this)
  for (j in lags:2) {
    lag_name <- paste0("DLVol_Lag_", j)
    prev_lag <- paste0("DLVol_Lag_", j - 1)
    if (lag_name %in% names(new_row) && prev_lag %in% names(new_row)) {
      new_row[1, lag_name] <- current_data[1, prev_lag]
    }
  }
  if ("DLVol_Lag_1" %in% names(new_row)) {
    new_row[1, "DLVol_Lag_1"] <- current_data[1, "DLVol_Lag_1"]
  }
  # Set current_data to new_row for next iteration
  current_data <- new_row
}
print(forecast_vals)
```

```
##  [1] 0.003838740 0.002957575 0.002932073 0.002931335 0.002931314 0.002931313
##  [7] 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313
## [13] 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313
## [19] 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313
## [25] 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313 0.002931313
```

```
# Transform Back to Price Level
last_log_price <- tail(ntfx_xts$Log_Close, 1)  # or use last actual log(close)
log_price_forecast <- cumsum(c(as.numeric(last_log_price), forecast_vals))[-1]
price_forecast <- exp(log_price_forecast)
plot(price_forecast, type = "l", main = "Forecasted Price", ylab = "Price", xlab = "Steps Ahead",
     cex.main = 0.8,
     cex.lab = 0.7,
     cex.axis = 0.6)
```


Forecasted Price
Price
1.02
1.06
0
5
10
15
20
25
30
Steps Ahead